

Vercel deployment

User

deployment with vercel h

Assistant

Looks like your message got cut off — “deployment with vercel h...” Could you finish that thought? In the meantime, quick check: your app is a Vite + React + TS project using Supabase (with a `.env` file for keys). Deploying to Vercel is straightforward, but I want to know exactly what you need before I write a tutorial.

Assistant

Good, I’ve got the style. Now writing the Vercel deployment tutorial matching this repo’s format.

Assistant

Tutorial’s ready. Key steps: create `vercel.json` yourself (rewrite rule so React Router routes don’t 404 on refresh), import the repo on vercel.com, re-enter your three env vars there (your `.env` is gitignored so Vercel never sees it), then deploy. Step 7 covers updating Supabase’s redirect URLs once you have the live domain.

User

[pasted a failed Vercel build log — 16 TypeScript errors: `import.meta.env` not typed, several unused imports/vars, `lodash/truncate` missing types]

Assistant

Diagnosed it: this build was against `main` (still the bare Assignment 1 code, per the “Initial Commit” log line) — none of the Supabase/OAuth work is on that branch, it’s sitting uncommitted or on `develop`. Two things to sort out:

Branch mismatch — either merge the real work into `main` and push, or switch Vercel’s Production Branch to `develop` in Project Settings → Git.

Real `tsc` errors, all because `vite dev` skips type-checking so they were never caught locally: - Missing `src/vite-env.d.ts` — explains 6 of the 16 errors (`import.meta.env` needs Vite’s client types). - `lodash/truncate` deep import doesn’t resolve under `"moduleResolution": "bundler"` — switch to a named import from `lodash` root. - 9 genuinely unused imports/consts, caught because `tsconfig.json` has `noUnusedLocals/noUnusedParameters` on.

Added a “Troubleshooting” section to `vercel-deployment-tutorial.md` with exact diffs for each, plus a heads-up that the real branch will hit a different (longer) error list once pushed — including an `AuthSession` type mismatch in `auth-api.ts` and old-style MUI `Grid item xs={}` props that the installed MUI version no longer accepts.

User

[pasted a second failed build log — 28 TypeScript errors across `auth-api.ts`, `supabase-client.ts`, `tmdb-api.ts`, `forgotPassword`, `movieCard`, MUI `Grid` usage, `moviesContext.tsx`, and more unused-declaration cleanups]

Assistant

Good progress — `vite-env.d.ts` and `lodash` fixes from round 1 landed cleanly (gone from this list). What’s left, added to the tutorial:

- `vite-env.d.ts` still missing — accounts for 8 of the 28 errors, same fix as before.
- `auth-api.ts` is dead code — nothing imports `loginUser/getSession/saveSession` anymore (auth went through Supabase). Its `AuthSession` type mismatch is leftover; delete the file or fix the shape.
- `React.SubmitEvent` isn't real — should be `React.FormEvent<HTMLFormElement>` in `forgotPassword/index.tsx`.
- MUI Grid v2 API — 7 errors across `movieCard`, `movieList`, `templateMovieListPage`, `templateMoviePage`. `item` is gone, `xs/sm/md/...` collapse into one `size` prop.
- 9 more unused-declaration cleanups — plus one worth a real look: `moviesContext.tsx`'s `addReview` takes `movie` but never uses it, even though nothing tags the pushed review with which movie it belongs to. Flagged as a possible actual gap, not just a lint nit.

All diffs appended to `vercel-deployment-tutorial.md` under “Troubleshooting round 2.” Run `npm run build` again after applying them — and settle the branch-mismatch issue from round 1 before pushing.

User

[pasted a third build log, now on the develop branch — 9 remaining TypeScript errors]

Assistant

Down to 9 from 28 — nice work. Both remaining issues trace back to fixes already in the tutorial:

1. `src/vite-env.d.ts` still isn't there (8 of 9 errors). Worth double-checking it actually got saved in `src/` on this `develop` branch and isn't sitting somewhere else or gitignored — it's the one fix that's resurfaced three rounds running.
2. `moviesContext.tsx:83` — the `movie` param on `addReview`, same call as before: `_movie` if intentionally unused, or wire it to `movie.id` if reviews should track which movie they're for.

Added as “round 3” to the tutorial file. Once those two are in, `npm run build` should exit clean.

User

[pasted a final single error — `addReview` call site missing the `movie` argument]

Assistant

Good news: `vite-env.d.ts` finally landed — that whole category is gone, down to this single error. What happened: the `movie` param got deleted from `addReview` instead of prefixed. But `MovieContextInterface` declares `addReview: (movie: BaseMovieProps, review: Review) => void` — two params — so the implementation has to keep both positionally, even if the first is unused. `reviewForm/index.tsx` already tags `review.movieId = movie.id` before calling `addReview`, so the param genuinely isn't needed inside the function itself — just put it back as intentionally-unused:

```
const addReview = useCallback(
-   ( review: Review) => {
+   (_movie: BaseMovieProps, review: Review) => {
    persist({ ...movieData, reviews: [...movieData.reviews, review] });
  },
  [movieData, persist]
);
```

That should be the last one — `npm run build` should exit clean after this.

User

[pasted a successful Vercel build log, but Commit 9c62273 on branch main — same as the very first attempt]

Assistant

This build is still against `main` at commit `9c62273` — same commit as the very first attempt. None of the fixes made (on `develop/feature/vercel_deployment`) have reached `main`, so this will fail with the same original 16 errors again. This is the branch-mismatch from round 1, still unresolved. Two ways to fix it:

Option A — point Vercel at the real branch: Vercel dashboard → project → Settings → Git → change Production Branch from `main` to `develop`. Fastest fix, no git needed.

Option B — bring `main` up to date:

```
git checkout main
git merge develop
git push origin main
```

Do this only if `main` should specifically become the production branch going forward.

Either way, don't wait on this build — it's going to fail on the same old code.

User

you almost made me mess up here

Assistant

Good catch — `main`'s single commit (“Initial Commit - Starting point is from end of assignment 1”) is the Assignment 1 checkpoint. Merging `develop` into it would have overwritten that reference point, which isn't something to touch even though it would've been a clean fast-forward with nothing lost mechanically. `main` should stay as-is.

Go with Option A: Vercel dashboard → project → Settings → Git → change Production Branch from `main` to `develop`. That's the only change needed — no git commands on `main` at all.

User

release with git flow

Assistant

This gives `main` a proper, deliberate release instead of the accidental merge from before: cut `release/1.0.0` off `develop`, bump the version, verify the build, then merge into `main` with `--no-ff` and a `v1.0.0` tag (and back into `develop` to keep them even). Once tagged, switch Vercel's Production Branch to `main` — same end state as before, but now versioned and reviewable instead of a blind merge.

One thing before starting: there's a stale `.git/index.lock` in the working copy — delete it first (unless a git GUI or another terminal is actively mid-operation) or the first command will fail.

User

[git error] fatal: Unable to create '`.git/index.lock`': File exists. Another git process seems to be running in this repository...

Assistant

That's the stale lock mentioned. Close any git GUI, VS Code Source Control panel, or other terminal that might be mid-operation, then delete it:

In Git Bash: `rm .git/index.lock` Or in PowerShell/CMD: `del .git\index.lock`

Then retry `git add -A -- ..`. If it fails again with the same error, check Task Manager for a stray `git.exe` process and end it before deleting the file.

User

[git flow release finish error] Error: failed to checkout target branch 'main' ... Error: a merge is already in progress for branch 'release/1.0.0'. Use `-continue` or `-abort`

Assistant

Found it — `.git/gitflow/state/merge.json` shows the extension's own "finish" process got stuck mid-merge, and that's what's blocking the second attempt.

Root cause of the very first failure: the release branch still has all ~65 files uncommitted (the "prep for vercel deployment" commits from before didn't actually include them — they've been riding along uncommitted through every branch switch since). `git flow` tried to `checkout main` as part of "finish" and Git refused because switching branches would've clobbered those uncommitted changes.

Fix, in order:

1. Cancel the stuck finish: `git flow release finish 1.0.0 --abort` (or delete `.git/gitflow/state/merge.json` directly if that errors too)
2. Commit the actual work on the release branch — this is the step that was skipped:

```
git add -A
git commit -m "release 1.0.0: build fixes and vercel deployment prep"
git status
```

Confirm the last line says "nothing to commit, working tree clean." 3. Retry finish: `git flow release finish 1.0.0` (may pop an editor for the tag/merge commit messages — save and close it, `:wq` in Vim, rather than closing the window) 4. Push everything — `git flow finish` doesn't push by default:

```
git push origin main --tags
git push origin develop
git push origin --delete release/1.0.0
```

User

<https://movies-app-assignment2-lj7j-one.vercel.app/>

Assistant

Deployment's live and serving the SPA shell, but that's a plain fetch — no JS execution, so I couldn't see if the app actually renders without a real browser. Yes — as long as `main` is set as the Production Branch in Vercel (Settings → Git), every push to `main` triggers an automatic rebuild and redeploy at that same URL, no manual redeploy step needed. A raw fetch shows the page loads (SPA shell titled "Vite + React + TS"), confirming the deploy succeeded. I tried to confirm further with the browser tool but the Chrome extension wasn't connected — install it and sign in for automatic checks, or open the link directly.

User

[screenshot of a git graph] How to fix this

Assistant

Looking at this graph, the release actually finished correctly — `main` is at `f182e3b8`, tagged `v1.0.0`, and matches `origin/HEAD`, and `develop` has the merge-back commit `db59e6a6`. That's exactly what a git-flow release finish should produce.

The one real gap: `origin/develop` is still stuck at `432dd6e4` (“prep for vercel deployment 2”) — local `develop` has moved ahead with the merge-back commit, but that hasn’t been pushed yet. Fix:

```
git checkout develop
git push origin develop
```

If there’s a specific error or symptom that prompted “how to fix this,” share it — nothing in the screenshot itself looks broken, want to make sure the actual problem is being fixed rather than just this stale-push gap.

User

git checkout develop git push origin develop explain this to me

Assistant

`git checkout develop` — switches the working directory to the `develop` branch. Likely still on `main` (from the release finish), so this moves HEAD back to `develop` and swaps files to match its latest commit (the merge-back commit `db59e6a6`). Nothing gets sent anywhere yet — purely local.

`git push origin develop` — uploads local `develop` to the `origin` remote (GitHub), updating `origin/develop` to match. `origin/develop` is stuck at the older commit `432dd6e4`; local `develop` has moved ahead (it has the merge from `main`) but GitHub doesn’t know that yet. This command tells GitHub about it.

Together: switch to the branch, then publish its current state to GitHub. Standard two-step whenever committing or merging locally and wanting the remote to catch up.